

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from google.colab import files

# ===== 1. LOAD + PREPROCESS DATA =====
print("Upload file Dry_Bean_Dataset.xlsx")
uploaded = files.upload()

df = pd.read_excel("Dry_Bean_Dataset.xlsx")

# Pisah X dan y
X = df.drop('Class', axis=1).values
y = df['Class'].values

# Label encode + One-hot encoding 7 kelas
le = LabelEncoder()
y = le.fit_transform(y)
y = np.eye(7)[y] # 7 neuron output

# Split 80:20 sesuai spek
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normalisasi manual
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

print(f"Train: {X_train.shape}, Test: {X_test.shape}\n")

# ===== 2. FUNGSI AKTIVASI + LOSS =====
def relu(z):
    return np.maximum(0, z)

def relu_deriv(z):
    return (z > 0).astype(float)

def softmax(z):
    exp_z = np.exp(z - np.max(z, axis=1, keepdims=True))
    return exp_z / exp_z.sum(axis=1, keepdims=True)

def mse_loss(y_pred, y_true):
    return np.mean((y_pred - y_true) ** 2)

# ===== 3. CLASS ANN FROM SCRATCH =====
class ANN_From_Scratch:
    def __init__(self, input_size, hidden1, hidden2, output_size):
        # ZERO INITIALIZATION sesuai spek
        self.W1 = np.zeros((input_size, hidden1))
        self.b1 = np.zeros((1, hidden1))
        self.W2 = np.zeros((hidden1, hidden2))
        self.b2 = np.zeros((1, hidden2))
        self.W3 = np.zeros((hidden2, output_size))
        self.b3 = np.zeros((1, output_size))

    def forward(self, X):
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = relu(self.z1)

```

```

        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = relu(self.z2)

        self.z3 = np.dot(self.a2, self.W3) + self.b3
        self.a3 = softmax(self.z3) # output
        return self.a3

def backward(self, X, y, lr):
    m = X.shape[0]

    # Output layer
    dz3 = self.a3 - y
    dW3 = np.dot(self.a2.T, dz3) / m
    db3 = np.sum(dz3, axis=0, keepdims=True) / m

    # Hidden layer 2
    da2 = np.dot(dz3, self.W3.T)
    dz2 = da2 * relu_deriv(self.z2)
    dW2 = np.dot(self.a1.T, dz2) / m
    db2 = np.sum(dz2, axis=0, keepdims=True) / m

    # Hidden layer 1
    da1 = np.dot(dz2, self.W2.T)
    dz1 = da1 * relu_deriv(self.z1)
    dW1 = np.dot(X.T, dz1) / m
    db1 = np.sum(dz1, axis=0, keepdims=True) / m

    # UPDATE BOBOT Mini-batch SGD sesuai spek lr=0.1
    self.W3 -= lr * dW3
    self.b3 -= lr * db3
    self.W2 -= lr * dW2
    self.b2 -= lr * db2
    self.W1 -= lr * dW1
    self.b1 -= lr * db1

def train(self, X, y, epochs=50, batch_size=16, lr=0.1):
    for epoch in range(epochs):
        # Shuffle data tiap epoch
        idx = np.random.permutation(X.shape[0])
        X_shuffle, y_shuffle = X[idx], y[idx]

        # MINI-BATCH SGD sesuai spek batch_size=16
        for i in range(0, X.shape[0], batch_size):
            X_batch = X_shuffle[i:i+batch_size]
            y_batch = y_shuffle[i:i+batch_size]

            self.forward(X_batch)
            self.backward(X_batch, y_batch, lr)

        # Hitung loss tiap 10 epoch
        if epoch % 10 == 0:
            pred = self.forward(X)
            loss = mse_loss(pred, y)
            print(f"Epoch {epoch}: Loss = {loss:.4f}")

def predict(self, X):
    pred = self.forward(X)
    return np.argmax(pred, axis=1)

# ===== 4. TRAINING =====
input_size = X_train.shape[1] # 16
hidden1 = 64
hidden2 = 32

```

```

output_size = 7

model = ANN_From_Scratch(input_size, hidden1, hidden2, output_size)

print("Mulai Training...")
model.train(X_train, y_train, epochs=50, batch_size=16, lr=0.1)

# ===== 5. EVALUASI =====
y_pred = model.predict(X_test)
y_true = np.argmax(y_test, axis=1)

accuracy = np.mean(y_pred == y_true) * 100
print(f"\nAkurasi Test: {accuracy:.2f}%")

```

Upload file Dry_Bean_Dataset.xlsx

Tidak ada file yang dipilih Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
 Saving Dry_Bean_Dataset.xlsx to Dry_Bean_Dataset (2).xlsx
 Train: (10888, 16), Test: (2723, 16)

```

Mulai Training...
Epoch 0: Loss = 0.1181
Epoch 10: Loss = 0.1181
Epoch 20: Loss = 0.1181
Epoch 30: Loss = 0.1180
Epoch 40: Loss = 0.1181

```

Akurasi Test: 24.64%

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from google.colab import files

# ===== 1. LOAD + PREPROCESS BINARY =====
print("Upload file Dry_Bean_Dataset.xlsx")
uploaded = files.upload()

df = pd.read_excel("Dry_Bean_Dataset.xlsx")

# BINARY CLASS: BOMBAY = 1, lainnya = 0
df['Class'] = (df['Class'] == 'BOMBAY').astype(int)

X = df.drop('Class', axis=1).values
y = df['Class'].values.reshape(-1, 1) # bentuk (n,1) buat binary

# Split 80:20 sesuai spek
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Normalisasi manual
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

print(f"Train: {X_train.shape}, Test: {X_test.shape}")

```

```

print(f"Proporsi kelas 1 di train: {y_train.mean():.2%}\n")

# ===== 2. FUNGSI AKTIVASI + LOSS =====
def sigmoid(z):
    z = np.clip(z, -500, 500) # biar nggak overflow
    return 1 / (1 + np.exp(-z))

def sigmoid_deriv(z):
    s = sigmoid(z)
    return s * (1 - s)

def binary_crossentropy(y_pred, y_true):
    eps = 1e-15 # biar nggak log(0)
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

# ===== 3. CLASS ANN FCL SIGMOID =====
class ANN_Binary:
    def __init__(self, input_size, hidden1, hidden2, output_size=1):
        # XAVIER/Glorot Initialization sesuai spek
        self.W1 = np.random.randn(input_size, hidden1) * np.sqrt(1/input_size)
        self.b1 = np.zeros((1, hidden1))
        self.W2 = np.random.randn(hidden1, hidden2) * np.sqrt(1/hidden1)
        self.b2 = np.zeros((1, hidden2))
        self.W3 = np.random.randn(hidden2, output_size) * np.sqrt(1/hidden2)
        self.b3 = np.zeros((1, output_size))

    def forward(self, X):
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = sigmoid(self.z1)

        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = sigmoid(self.z2)

        self.z3 = np.dot(self.a2, self.W3) + self.b3
        self.a3 = sigmoid(self.z3) # output binary 0-1
        return self.a3

    def backward(self, X, y, lr=0.01):
        m = X.shape[0]

        # Output layer - deriv BCE + Sigmoid
        dz3 = self.a3 - y
        dW3 = np.dot(self.a2.T, dz3) / m
        db3 = np.sum(dz3, axis=0, keepdims=True) / m

        # Hidden layer 2
        da2 = np.dot(dz3, self.W3.T)
        dz2 = da2 * sigmoid_deriv(self.z2)
        dW2 = np.dot(self.a1.T, dz2) / m
        db2 = np.sum(dz2, axis=0, keepdims=True) / m

        # Hidden layer 1
        da1 = np.dot(dz2, self.W2.T)
        dz1 = da1 * sigmoid_deriv(self.z1)
        dW1 = np.dot(X.T, dz1) / m
        db1 = np.sum(dz1, axis=0, keepdims=True) / m

        # UPDATE BOBOT Mini-batch SGD lr=0.01
        self.W3 -= lr * dW3
        self.b3 -= lr * db3
        self.W2 -= lr * dW2
        self.b2 -= lr * db2
        self.W1 -= lr * dW1

```

```

self.b1 -= lr * db1

def train(self, X, y, epochs=100, batch_size=32, lr=0.01):
    for epoch in range(epochs):
        # Shuffle data tiap epoch
        idx = np.random.permutation(X.shape[0])
        X_shuffle, y_shuffle = X[idx], y[idx]

        # MINI-BATCH SGD batch_size=32
        for i in range(0, X.shape[0], batch_size):
            X_batch = X_shuffle[i:i+batch_size]
            y_batch = y_shuffle[i:i+batch_size]

            self.forward(X_batch)
            self.backward(X_batch, y_batch, lr)

        # Print loss tiap 20 epoch
        if epoch % 20 == 0:
            pred = self.forward(X)
            loss = binary_crossentropy(pred, y)
            print(f"Epoch {epoch}: Loss = {loss:.4f}")

    def predict(self, X, threshold=0.5):
        pred = self.forward(X)
        return (pred > threshold).astype(int)

# ===== 4. TRAINING =====
input_size = X_train.shape[1] # 16
hidden1 = 32
hidden2 = 16

model = ANN_Binary(input_size, hidden1, hidden2)

print("Mulai Training...")
model.train(X_train, y_train, epochs=100, batch_size=32, lr=0.01)

# ===== 5. EVALUASI =====
y_pred = model.predict(X_test)
accuracy = np.mean(y_pred == y_test) * 100
print(f"\nAkurasi Test Binary: {accuracy:.2f}%")

```

Upload file Dry_Bean_Dataset.xlsx

Tidak ada file yang dipilih Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving Dry_Bean_Dataset.xlsx to Dry_Bean_Dataset (3).xlsx

Train: (10888, 16), Test: (2723, 16)

Proporsi kelas 1 di train: 3.84%

Mulai Training...

Epoch 0: Loss = 0.1843

Epoch 20: Loss = 0.0765

Epoch 40: Loss = 0.0236

Epoch 60: Loss = 0.0109

Epoch 80: Loss = 0.0066

Akurasi Test Binary: 100.00%

```

from sklearn.metrics import classification_report, confusion_matrix

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Bukan BOMBAY', 'E

```

Confusion Matrix:

```

[[2619    0]
 [    0  104]]

```

Classification Report:

	precision	recall	f1-score	support
Bukan BOMBAY	1.00	1.00	1.00	2619
BOMBAY	1.00	1.00	1.00	104
accuracy			1.00	2723
macro avg	1.00	1.00	1.00	2723
weighted avg	1.00	1.00	1.00	2723

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from google.colab import files

# ===== 1. LOAD + PREPROCESS MULTICLASS =====
print("Upload file Dry_Bean_Dataset.xlsx")
uploaded = files.upload()

df = pd.read_excel("Dry_Bean_Dataset.xlsx")

# Multiclass: 7 kelas asli, jadiin 0-6
kelas = df['Class'].astype('category').cat.codes.values
X = df.drop('Class', axis=1).values
y = kelas

# One-hot encoding manual buat 7 kelas
num_classes = len(np.unique(y))
y_onehot = np.eye(num_classes)[y]

# Split 80:20 sesuai spek
X_train, X_test, y_train, y_test = train_test_split(
    X, y_onehot, test_size=0.2, random_state=42, stratify=y
)
y_test_label = np.argmax(y_test, axis=1) # buat evaluasi akhir

# Normalisasi manual
mean = X_train.mean(axis=0)
std = X_train.std(axis=0)
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

print(f"Train: {X_train.shape}, Test: {X_test.shape}")
print(f"Jumlah kelas: {num_classes}\n")

# ===== 2. FUNGSI AKTIVASI + LOSS =====
def tanh(z):
    return np.tanh(z)

```

```

def tanh_deriv(z):
    return 1 - np.tanh(z)**2

def softmax(z):
    exp_z = np.exp(z - np.max(z, axis=1, keepdims=True)) # stabil biar nggak
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)

def categorical_crossentropy(y_pred, y_true):
    eps = 1e-15
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.mean(np.sum(y_true * np.log(y_pred), axis=1))

# ===== 3. CLASS ANN FCL TANH =====
class ANN_Multiclass:
    def __init__(self, input_size, hidden1, hidden2, output_size):
        # RANDOM UNIFORM INIT sesuai spek. Range [-0.5, 0.5]
        limit = 0.5
        self.W1 = np.random.uniform(-limit, limit, (input_size, hidden1))
        self.b1 = np.zeros((1, hidden1))
        self.W2 = np.random.uniform(-limit, limit, (hidden1, hidden2))
        self.b2 = np.zeros((1, hidden2))
        self.W3 = np.random.uniform(-limit, limit, (hidden2, output_size))
        self.b3 = np.zeros((1, output_size))

    def forward(self, X):
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = tanh(self.z1) # hidden 1: Tanh

        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = tanh(self.z2) # hidden 2: Tanh

        self.z3 = np.dot(self.a2, self.W3) + self.b3
        self.a3 = softmax(self.z3) # output: Softmax
        return self.a3

    def backward(self, X, y, lr=0.001):
        m = X.shape[0]

        # Output layer - deriv CCE + Softmax = y_pred - y_true
        dz3 = self.a3 - y
        dW3 = np.dot(self.a2.T, dz3) / m
        db3 = np.sum(dz3, axis=0, keepdims=True) / m

        # Hidden layer 2
        da2 = np.dot(dz3, self.W3.T)
        dz2 = da2 * tanh_deriv(self.z2)
        dW2 = np.dot(self.a1.T, dz2) / m
        db2 = np.sum(dz2, axis=0, keepdims=True) / m

        # Hidden layer 1
        da1 = np.dot(dz2, self.W2.T)
        dz1 = da1 * tanh_deriv(self.z1)
        dW1 = np.dot(X.T, dz1) / m
        db1 = np.sum(dz1, axis=0, keepdims=True) / m

        # UPDATE BOBOT Mini-batch SGD lr=0.001
        self.W3 -= lr * dW3
        self.b3 -= lr * db3
        self.W2 -= lr * dW2
        self.b2 -= lr * db2
        self.W1 -= lr * dW1
        self.b1 -= lr * db1

```

```

def train(self, X, y, epochs=200, batch_size=64, lr=0.001):
    for epoch in range(epochs):
        # Shuffle data tiap epoch
        idx = np.random.permutation(X.shape[0])
        X_shuffle, y_shuffle = X[idx], y[idx]

        # MINI-BATCH SGD batch_size=64
        for i in range(0, X.shape[0], batch_size):
            X_batch = X_shuffle[i:i+batch_size]
            y_batch = y_shuffle[i:i+batch_size]

            self.forward(X_batch)
            self.backward(X_batch, y_batch, lr)

        # Print loss tiap 40 epoch biar nggak spam
        if epoch % 40 == 0:
            pred = self.forward(X)
            loss = categorical_crossentropy(pred, y)
            print(f"Epoch {epoch}: Loss = {loss:.4f}")

def predict(self, X):
    pred = self.forward(X)
    return np.argmax(pred, axis=1)

# ===== 4. TRAINING =====
input_size = X_train.shape[1] # 16
hidden1 = 32
hidden2 = 16
output_size = num_classes # 7

model = ANN_Multiclass(input_size, hidden1, hidden2, output_size)

print("Mulai Training...")
model.train(X_train, y_train, epochs=200, batch_size=64, lr=0.001)

# ===== 5. EVALUASI =====
y_pred = model.predict(X_test)
accuracy = np.mean(y_pred == y_test_label) * 100
print(f"\nAkurasi Test Multiclass: {accuracy:.2f}%")

```

Upload file Dry_Bean_Dataset.xlsx

Tidak ada file yang dipilih Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving Dry_Bean_Dataset.xlsx to Dry_Bean_Dataset (4).xlsx

Train: (10888, 16), Test: (2723, 16)

Jumlah kelas: 7

Mulai Training...

Epoch 0: Loss = 1.6356

Epoch 40: Loss = 0.4869

Epoch 80: Loss = 0.3285

Epoch 120: Loss = 0.2732

Epoch 160: Loss = 0.2479

Akurasi Test Multiclass: 91.81%

#Nama : Barep Purwo Agosto
Prodi : STI
UNIVERSITAS DARUNNAJAH